

A large, faint watermark of the ISTAD logo is centered in the background. It is a circular emblem with a blue outer ring containing the text 'INSTITUTE OF SCIENCE AND TECHNOLOGY ADVANCED DEVELOPMENT' and Khmer text. Inside the ring is a blue circle with binary code '01010011 01010100 01000001 0100100' and a central graphic of a stylized atom or network.

DevOps

Linux Command with Bash Script

Getting Started with Bash Script

- **A Bash Shell Script** is a plain text file containing a set of various commands that we usually type in the command line. It is used to automate repetitive tasks on Linux filesystem. It might include a set of commands, or a single command, or it might contain the hallmarks of imperative programming like loops, functions, conditional constructs, etc.
- **Shebang(hashbank)** is a special character sequence in a script file that specifies which program should be called to run the script. The shebang is always on the first line of the file, and is composed of the characters **#!** followed by the path to the interpreter program.

Getting Started with Bash Script

- `$ sudo vim hello-world.sh`
 - extension can be txt, doc, bash...

```
#!/bin/bash
```

```
echo "hello world"
```

```
#!/usr/bin/env bash
```

```
echo "hello world"
```

- To run the file script:
 - `$ bash hello-world.sh` **or** `$ sh hello-world.sh`
 - `./hello-world.sh` **but** `$ chmod +x hello-wold.sh`

echo command

- **echo** is used to display line of text/string that are passed as an argument.

```
#!/bin/bash  
echo -n "What is your name? "  
read name  
echo "Your name is $name"  
echo -e "Hello \n world"
```

```
seng@ip-172-31-17-136:~$ bash echo-script.sh  
What is your name? dara  
Your name is dara  
Hello  
world
```

echo command

```
#!/bin/bash  
username="Sok sao"  
echo "${#username}" # length of String  
echo "${username^^}" # Convert to UPPERCASE  
echo "${username,,}" # Convert to lowercase
```

```
[seng@ip-172-31-17-136:~$ bash echo-2.sh  
7  
SOK SAO  
sok sao
```

echo command

```
#!/bin/bash
```

```
var="Today are Monday"
```

```
echo "${var/are/IS}" # Find and replace (Not overwrite $var)
```

```
echo "${var:2:3}" # 2 is index; 3: take 3 characters from index 2
```

```
echo "${var:5}" # 5: take from index 5 to the end
```

```
seng@ip-172-31-17-136:~$ bash echo-3.sh  
Today IS Monday  
day  
are Monday
```

date command

- **date** used to display the system date and time

```
#!/bin/bash
echo "What day is today? $(date)"
echo "What day is today? $(date '+%F@%T')"
echo "What is the date? $(date '+%d-%m-%Y')"
echo "What time is it? $(date '+%H:%M:%S')"
echo "Day & Month $(date '+%a-%A & %b-%B')"
```

```
[seng@ip-172-31-17-136:~$ bash my-shell.bash
What day is today? Wed Nov  2 16:06:55 UTC 2022
What day is today? 2022-11-02@16:06:55
What is the date? 02-11-2022
What time is it? 16:06:55
Day & Month Wed-Wednesday & Nov-November
```

cut command

- **cut** is a command for cutting out the sections from each line of files and writing the result to standard output.
- `$ vim cut-script.txt`

Lorem ipsum, in graphical and textual context.

- `$ cut -d " " -f 2 cut-script.txt`
Find delimiter “space” and take the 2nd field “-f”

```
seng@ip-172-31-17-136:~$ cut -d " " -f 2 cut-script.txt  
ipsum,
```

cut command

- `$ cut -c 1 cut-script.txt`
Show the first character in every line
- `$ cut -c 2- cut-script.txt`
Show the text from second characters in every line
- `$ cut -c -3 cut-script.txt`
Show the text from 1st characters to 3rd in every line
- `$ cut -c 2-8,10 cut-script.txt`
Show the text from 2nd character to 8th and 10th in every line

tr command

- **tr** is a command line utility for replacing or deleting characters.
- `$ vim cut-script.txt`

Lorem ipsum, in graphical and textual context.

- `$ cut -d " " -f2 cut-script.txt | tr -s "," "$" # -s is optional`

```
seng@ip-172-31-17-136:~$ cut -d " " -f2 cut-script.txt | tr ", " "$"  
ipsum$
```

tr command

- `$ cat cut-script.txt | tr -d "[,.]"` # Delete all “,” and “.” in text
- `$ tr -d "[,.]< cut-script.txt`

```
[seng@ip-172-31-17-136:~$ cat cut-script.txt | tr -d "[,.]"  
Lorem ipsum in graphical and textual context  
[seng@ip-172-31-17-136:~$ tr -d "[,.]< cut-script.txt  
Lorem ipsum in graphical and textual context
```

tr command

- `$ tr '[:lower:]' '[:upper:]' < filename.txt # Convert to UPPERCASE`
- `$ tr '[a-z]' '[A-Z]' < filename.txt # Convert to UPPERCASE`
- `$ echo "Welcome To tesla" | tr -d 'tT' # Delete letter "t,T"`
- `$ echo "my ID is 73535" | tr -d [:digit:] # Delete only number`
- `$ echo "my ID is #73535" | tr -cd [:digit:]`
`# Delete everything beside number`
- `$ echo "my ID is #73535" | tr -d [:alpha:] # Delete only characters`

File Descriptor

- **File Descriptor** is a process-unique identifier (handle) for a file or other input/output resource.

```
$ read -u 0 name # -u 0 is optional
```

Descriptor	Explanation
0	Represents standard input.
1	Represents standard output.
2	Represents standard error.

- `$ echo "hey" 1>filename.txt # > create or overwrite file`
- `$ echo "bye" 1>>filename.txt # >> create or append file`
- `$ asdf 1>succ.txt 2>err.txt`
`# Command not error write into succ.txt. If error, write into err.txt.`

File Descriptor

- `$ ls 1>filename.txt 2>filename.txt`
- `$ ls 1>filename.txt 2>&1`
- `$ ls &>filename.txt`

Those 3 command lines above will show the same result.

The result of the "ls" command will always be saved to the file "filename.txt," whether an error occurred or not.

more, head, tail command

- more, head, tail
 - `$ cat -n /etc/passwd | more -2` # Show text from 1st to 2nd line
 - `$ cat /etc/passwd | more +4` # Show text from 4th to the end of line
 - `$ head filename.txt` # Show the first 10 lines(Default)
 - `$ head -20 filename.txt` # Show the first 20 lines
 - `$ tail filename.txt` # Show the last 10 lines(Default)
 - `$ tail -20 filename.txt` # Show the last 20 lines

Line range

- How to show the text from line 17th to line 20th?
 - `head -20 filename.txt | tail -4`
 - `awk 'NR>=17 && NR<=20 {print}' filename.txt` # NR(Number of Record)
 - `sed -n '17,20p' filename.txt`

grep command

- **grep** is a command-line utility for searching plain-text data sets for lines that match a regular expression.
- `$ grep -w -n 'the' filename.txt # -w:exact word "the"; -n:line number`
- `$ grep -i "the" filename1.txt filename2.txt # -i:ignore case`
- `$ grep -v 'the' filename.txt # -v:Show all lines has no word "the"`
- `$ grep -o 'the' filename.txt # -o:Show only word "the" when match`
- `$ grep -c 'the' filename.txt # Count when match the result`
- `$ grep -A 1 'the' filename.txt # -A 1: Show one line after`
- `$ grep -B 1 'the' filename.txt # -B 1: Show one line before`
- `$ grep -C 1 'the' filename.txt # -C 1: Show one line before and after`

grep command

- `$ grep -E -i "the|farm" filename.txt`
Search “the” and “farm” with -i: ignore case; -E:regex
- `$ grep -E "^the" filename.txt` # Find “the” only start at beginning
- `$ grep -E "the$" filename.txt` # Find “the” at the last of each line
- `$ grep -E "^the$" filename.txt` # Find each line has only a word “the”
- `$ grep -E "t..s" filename.txt` # Find the word “t**s” with 4 characters
- `$ grep -E "i?s" filename.txt`
Find the word which has zero “i” or one “i” and “s”
- `grep -E "i*s" filename.txt` #
Find the word which has zero “i” or more than one “i” and “s”

grep command

- `$ grep -E "A|a" filename.txt` or `$ grep -E "[Aa]" filename.txt`
Find the word which has letter "A" or "a"
- `$ grep -E "[a-zA-Z]" filename.txt`
Find the word which has letter(lowercase) from "a" to "d" and letter (UPPERCASE) from "A" to "Z"
- `grep -E "^[TI]" filename.txt`
Find the word that start with "T" or "I" in the beginning
- `grep -E "[^ti]" filename.txt`
Find the word that doesn't has "t" or "i" in each line

grep command with Character Class

- `$ grep -E "es{2, 4}" filename.txt`
Find the word that has “ess” $\Rightarrow$ `s{2}` = `s x 2` or “essss” `s` = 4 times
- `grep -E "[[:upper:]]" filename.txt` # Find the UPPERCASE word
- `grep -E "[[:lower:]]" filename.txt` # Find the lowercase word
- `grep -E "[[:digit:]]" filename.txt` # Find the number
- `grep -E "[[:alpha:]]" filename.txt` # Find the word as alphabet
- `grep -E "[[:alnum:]]" filename.txt` # Find the word as
alphabet+number
- `grep -E "[[:blank:]]" filename.txt` # Find the space and tab
- `grep -E "[[:space:]]" filename.txt` # Find the space, tab, return...

Multiblock

```
#!/bin/bash  
cat << EOF > my-file.txt  
This is first line.  
This is second line.  
This is third line.  
EOF
```

```
#!/bin/bash  
echo "This is a sky.  
This is a book.  
The sun is shining."  
| tee -a file.txt
```

```
seng@ip-172-31-17-136:~$ cat my-file.txt  
This is first line.  
This is second line.  
This is third line.
```

Special Shell Variable

`$0` Filename

`$1` First parameter

`${10}` 10th parameter should be in curly bracket

`$*` or `$@` show all parameter values

`$?` Read exit status code of the last command

`$#` Number of parameter

Exit status code

0 is exit code success

1 or greater is exit code failed

```
seng@ip-172-31-17-136:~$ test 3 -eq 4  
seng@ip-172-31-17-136:~$ echo $?  
1
```

Conditional Operator

```
-eq : equal  
-ne : not equal  
-ge : greater than and equal  
-gt : greater than  
-le : less than and equal  
-lt : less than  
$ [[ ! 3 -lt 3 ]]  
echo $? => Result: 0
```

```
$ fruit="apple"  
$ [[ -z $fruit ]]  
Result: 1  
-z: length of string must be zero  
$ [[ -n $fruit ]]  
Result: 0  
-n: the length of string must be  
not null - existing
```

Chaining Operator ; && ||

. ; $\Rightarrow$ if; if

\$ which docker; docker -v # Not related between each statement

. && $\Rightarrow$ if{ if }

\$ which dockers &>/dev/null && docker -v # 2nd is worked until 1st is succeed

. || $\Rightarrow$ if{} else{}

\$ which dockers &>/dev/null || docker -v # 2nd is worked until 1st is failed

. && ||

\$ which dockers &>/dev/null && docker -v || echo "Another choice!"

Case Conditional

```
#!/bin/bash
read -p "Num1: " a
read -p "Num2: " b
read -p "Please select option: P: Plus, M: Minus, m: multiply, D: Divid: "
form
case $form in
    #Use case: 1) or [0-9]) or [a-z])
    "P" | "p")
        echo "You choose: $form"
        echo "$a plus $b is $(( $a + $b ))";;
    "M")
        echo "You choose: $form"
        echo "$a minus $b is $(( $a - $b ))";;
```

Case Conditional

```
"m")
    echo "You choose: $form"
    echo "$a multiply by $b is $(bc<<<"scale=2;$a*$b")";;
"D" | "d")
    echo "You choose: $form"
    echo "$a divide $b is $(bc<<<"scale=2;$a/$b")";;
*)
    echo "Please choose valid choices!";;
esac
```

```
seng@ip-172-31-17-136:~$ bash case-script.sh
Num1: 10
Num2: 3
Please select option: P: Plus, M: Minus, m: multiply, D: Divid: D
You choose: D
10 divide 3 is 3.33
```

If-else Conditional

```
if docker -v &>/dev/null;
then
    echo "This is docker version: $(docker -v)"
fi
*** OR ***
docker -v &>/dev/null
if [[ $? -eq 0 ]]; then
    echo "This is docker version: $(docker -v)"
fi
```

If-else Conditional

```
----- with -a "and" -----  
if [ 0 -eq 0 -a 1 -eq 1 ]; then  
    echo "This is docker version: $(docker -v)"  
fi  
  
----- with -o "or" -----  
if [ 0 -eq 1 -o 1 -eq 1 ]; then  
    echo "This is docker version: $(docker -v)"  
fi  
  
----- Use (()) instead of [[]] for number -----  
read -p "Are you teenager? " answer  
if (( $answer >= 18 )); then  
    echo "Yes, I am teenager."  
else  
    echo "No, I am not old enough."  
fi
```

If-else Conditional “&&” “||”

```
read -p "What is your age? " answer
if [[ $answer -ge 1 ]] && [[ $answer -lt 18 ]]; then
    echo "I am child."
elif [[ $answer -ge 18 ]] && [[ $answer -lt 50 ]]; then
    echo "I am adult."
elif [[ $answer -ge 50 ]] && [[ $answer -lt 90 ]]; then
    echo "I am old."
else
    echo "I am very very old."
fi
```

For Loop

```
for item in 1 2 3 4 5
do
    echo "This is item: $item"
done
#C-Language Type
for((i=1;i<=5;i++))
do
    echo "This is i: $i"
done
for item in $(seq 1 5)
do
    echo "This is item: $item"
done
```

```
seng@ip-172-31-17-136:~$ bash for-script.sh
This is item: 1
This is item: 2
This is item: 3
This is item: 4
This is item: 5
This is i: 1
This is i: 2
This is i: 3
This is i: 4
This is i: 5
This is item: 1
This is item: 2
This is item: 3
This is item: 4
This is item: 5
```

While Loop

```
count=1
while [[ $count -lt 5 ]]
do
    echo "Count is $count"
    ((count++))
done
```

```
seng@ip-172-31-17-136:~$ bash while-script.sh
Count is 1
Count is 2
Count is 3
Count is 4
```

sed command

- **sed** stands for **stream editor** and it can perform lots of functions on file like searching, find and replace, insertion or deletion.

#View

- `$ sed -n '1p' passwd-file.txt # Show text the first line`
- `$ sed -n '$p' passwd-file.txt # Show text the last line`
- `$ sed -n '3,10p' passwd-file.txt # Show line 3 to line 10`
- `$ sed -n '3,+7p' passwd-file.txt # Show line 3 to line 10 (3+7)`
- `$ sed -n '2~5p' passwd-file.txt # Show from line 2 and skip 4 lines...`
- `$ sed '10d' passwd-file.txt # Show all lines except line 10th`
- `$ sed '10,20d' passwd-file.txt # Show all lines except line 10th-20th`

sed command

- `$ sed '10,+2d' passwd-file.txt` # Show all lines except line 10th-12th

#Delete

- `$ sed -i '10,$d' passwd-file.txt`
-i:insert(overwrite file); \$:last line
delete from 10th line to the last line
- `$ sed -i.backup '10,$!d' passwd-file.txt`
-i.backup:create new file as backup
delete from 9th line to the 1st line(not include 10th line)

sed command

#Search

- `$ sed -n '/daemon/p' passwd-file.txt` # Show all result with “daemon”
`$ sed '/daemon/!d' passwd-file.txt`
- `$ sed -n '/[Tt]he/p' filename.txt` # Show the match “The” or “the”
`$ sed -n -e '/The/p' -e '/the/p' filename.txt`

#Search and Replace

- `$ sed -i 's/is/@/' filename.txt`
Replace “is” to “@” only the first found in each line
- `$ sed -i 's/is/@/gi' filename.txt` # gi: global and ignore case
Replace “is” to “@” when found in each line

sed command

- `$ sed -i 's/is/@/2' filename.txt`
Replace “is” to “@” when only found the 2nd time in each line
- `$ sed -i '/is/s/farm/@/g' filename.txt`
Search for “is” and replace “farm” to “@” in each line
- `$ sed -i '1i No Name Gender Phone' filename.txt`
insert “No Name Gender Phone” in the first line
1a : after the first line
\$a : At the end of all lines

sed command

- `$ sed -i '/seng/i 0 dara M 123123' filename.txt`
insert "0 dara M 123123" before line 'seng'
- `$ sed -i '0, /seng/a 2 dara M 123123' filename.txt`
insert "2 dara M 123123" after found line 'seng'
if "0," is absent, it will add in each line when found the match

sed command

#Regex

```
$ sed -n '/[a-zA-Z]/p' filename.txt # only character from a to z or A  
to Z $ sed -n '/^[a-z]/p' filename.txt # every line word start with a  
to z
```

```
$ sed -n '/[a-z]$/p' filename.txt # every line word finish with a to z
```

```
$ sed -n '/^apple$/p' filename.txt # find line that has only word
```

```
"apple" $ sed -n '/^[^0-9]/p' filename.txt
```

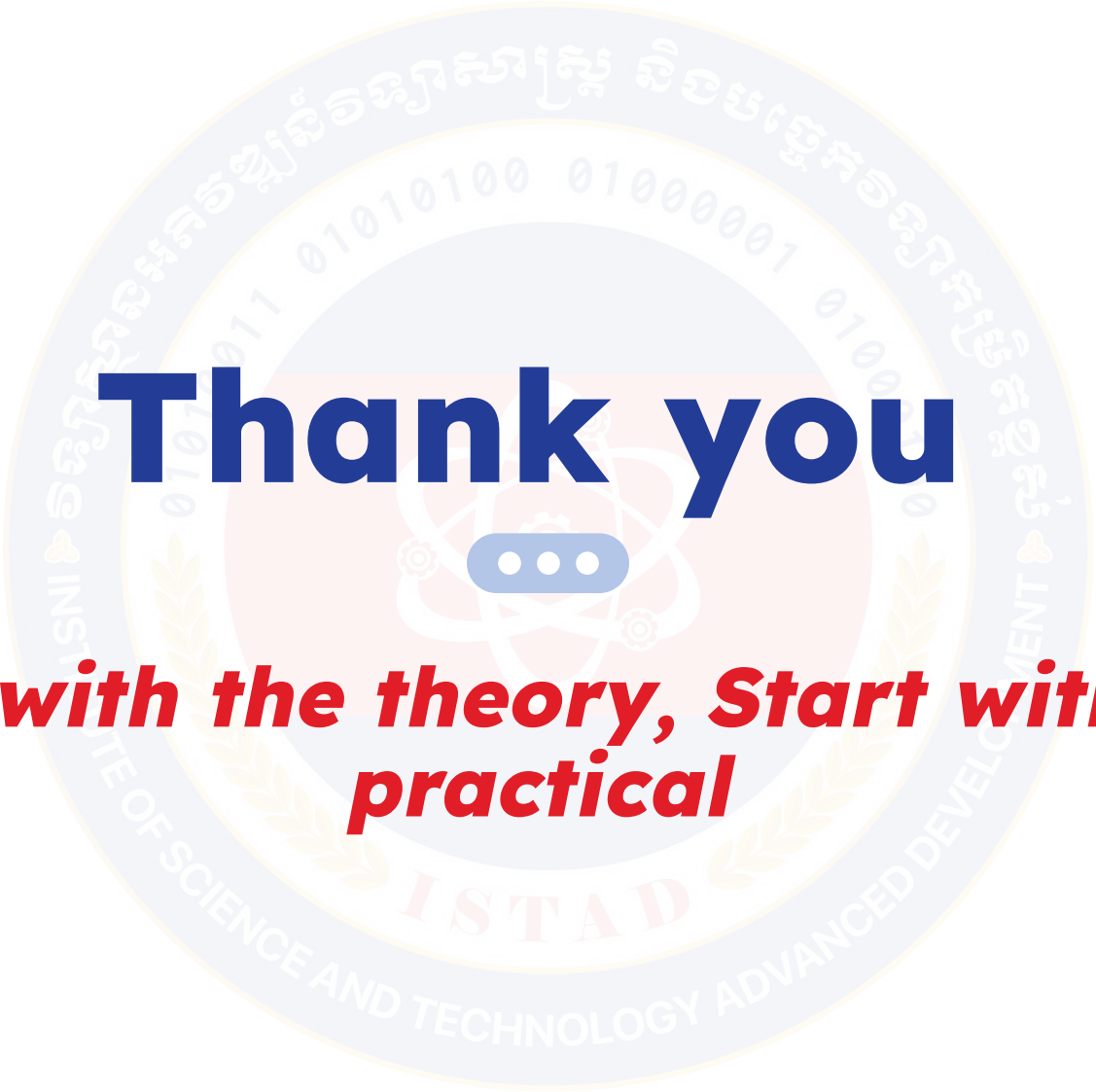
```
# every line not start with 0 to 9 = beside number
```

sed command

- `$ sed -i 's/\(image:.*:\)\(.*\)\/\1latest/' docker-compose.yaml`
We divide 2 group: 1st is `\(image:.*:\)`, 2nd is `\(.*\)`
We change to `\1latest` : `\1` is 1st group and 2nd is latest
Example: `image: postgres:13.5 => image: postgres:latest`
- `sed -i 's+postgres:.*+postgres:1.1.1+' docker-compose.yaml`
- `sed -i 's/postgres:.*\/postgres:1.1.1/' docker-compose.yaml`

yq command

- `$ sed -i 's/\(image:.*:\)\(.*\)\/\1latest/' docker-compose.yaml`
We divide 2 group: 1st is `\(image:.*:\)`, 2nd is `\(.*\)`
We change to `\1latest` : `\1` is 1st group and 2nd is latest
Example: `image: postgres:13.5 => image: postgres:latest`
- `sed -i 's+postgres:.*+postgres:1.1.1+' docker-compose.yaml`
- `sed -i 's/postgres:.*\/postgres:1.1.1/' docker-compose.yaml`

A large, faint watermark of the ISTAD logo is centered in the background. It is a circular emblem with a blue outer ring containing the text 'INSTITUTE OF SCIENCE AND TECHNOLOGY ADVANCED DEVELOPMENT' and Khmer text. Inside the ring is a red circle with a white atomic symbol and binary code '01010100 01000001 01000001' around it. The word 'ISTAD' is written in red at the bottom of the inner circle.

Thank you

Begin with the theory, Start with the practical